INDUSTRIAL TRAINING REPORT

on

BANK ACCOUNT MANAGEMENT SYSTEM USING JAVA

A Conceptual Case Study

by

Mohd Najeeb [Roll No.: _____]

Section 2

Submitted on Partial Fulfillment of the Requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

from

EWB EDUTECH PRIVATE LIMITED
(EWB COURSES)

Submitted to

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING & TECHNOLOGY

UNIVERSITY OF LUCKNOW, LUCKNOW

November, 2026

DECLARATION

I, Mohd Najeeb, a student of B.Tech Computer Science and Engineering, Section 2, University of Lucknow, hereby declare that the report entitled Bank Account Management System Using Java: A Conceptual Case Study has been prepared for submission as my Summer Internship Report.

The report is based on the learning context of my one-month Java Programming internship at EWB EDUTECH PRIVATE LIMITED (EWB Courses). It presents a technology-based academic case study and explains the application of Java concepts through a simple banking problem.

I understand that the project is conceptual. It does not include a working banking application, a connected database or real customer records. The interface layouts are illustrative drawings, and the numerical examples and charts use fictional learning data. These items are not presented as screenshots, company records or experimental results.

Sources used for technical explanations and the prescribed submission context have been acknowledged in the references. The supplied internship certificate has been reproduced without alteration. No confidential company information is included, and no company NDA is asserted in this report.

I take responsibility for reviewing the final document, adding my university roll number and signing this declaration before submission.

Name: Mohd Najeeb Roll Number: _____ Section: 2 Signature: _____ Date: _____ Place: Lucknow

TRAINING / INTERNSHIP CERTIFICATE

Colour reproduction of the original certificate supplied by the student.



Certificate ID: EWB/INT/2026/9765
Issue Date: 16 July 2026
Programme: Java Programming One Month

Print this page in colour. The certificate image has not been edited or recoloured.

ABSTRACT

This report presents a simple conceptual case study of a Bank Account Management System using Java. It has been prepared in connection with the student's one-month Java Programming internship at EWB EDUTECH PRIVATE LIMITED (EWB Courses). The study explains how basic banking activities account opening, deposits, withdrawals, balance enquiries and transaction history can be organised through clear information structures and programming rules.

The report follows the technology-based training approach permitted by the departmental instructions. Java platform concepts, classes and objects, encapsulation, inheritance, data types, control flow, validation, exceptions, collections and database connectivity are explained in straightforward language and related to the selected problem. A manual account-handling process is used as the problem context, followed by functional and non-functional requirements and a proposed layered design.

The application of the technology is demonstrated through use-case diagrams, flowcharts, entity relationship and class diagrams, conceptual interface layouts, responsibility tables, worked balance calculations and charts. Two separate illustrative datasets are used: a six-month activity-count table and a five-transaction ledger for a fictional account. All names, account numbers, numerical values and interface drawings are academic examples.

No working banking application, live database, real customer information or confidential company data is included. The interface layouts are not screenshots, and the charts are not measured results from an implemented system. The report therefore demonstrates design understanding and practical reasoning without claiming implementation, testing or production security. It concludes with limitations and a staged path for future supervised development.

Keywords: Java; bank account management; object-oriented programming; conceptual design; validation; illustrative data.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Department of Computer Science and Engineering, Faculty of Engineering & Technology, University of Lucknow, for including practical training and report preparation in the B.Tech learning process.

I am thankful to Dr. Sakshi Srivastava, the designated faculty member for Section 2 report submission, and to the department's faculty members for their guidance and for providing the instructions used to organise this report.

I thank EWB EDUTECH PRIVATE LIMITED (EWB Courses) for the Java Programming learning opportunity. The training provided the context for reviewing programming concepts and applying them to a relevant academic case study.

I also appreciate the support of my family and friends during the preparation of this report. Their encouragement helped me organise the work and complete the documentation.

Finally, I acknowledge the authors and organisations whose official documentation and books are listed in the references. Those sources supported the technical explanations. The banking examples, sample data and conceptual diagrams were prepared as academic illustrations for this report.

Mohd Najeeb B.Tech Computer Science and Engineering Section 2 University of Lucknow

TABLE OF CONTENTS

Title and page references

Declaration	ii
Training / Internship Certificate	iii
Abstract	iv
Acknowledgement	v
List of Tables	viii
List of Figures	ix
Abbreviations and Notation	x
CHAPTER 1: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Bank Accounts and Basic Operations	2
1.3 Need for a Simple Conceptual Study	3
1.4 Scope and Organisation of the Report	4
CHAPTER 2: OBJECTIVE OF THE PROJECT	5
2.1 Main Objective	5
2.2 Specific Objectives and Supporting Evidence	6
2.3 Expected Learning Outcomes and Boundaries	7
CHAPTER 3: OVERVIEW OF THE COMPANY AND TRAINING	8
3.1 Profile of the Training Organisation	8
3.2 Training Duration and Learning Themes	9
3.3 Technology-Based Training and the Case-Study Route	10
3.4 Professional Practices and Documentation Habits	11
CHAPTER 4: OVERVIEW OF TECHNOLOGY	12
4.1 Java and the Java Platform	12
4.2 Classes, Objects and Methods	13
4.3 Encapsulation, Abstraction and Access Control	14
4.4 Inheritance and Polymorphism	15
4.5 Variables, Data Types and Money Values	16
4.6 Control Flow and Method Organisation	17
4.7 Validation and Exception Handling	18
4.8 Collections, Generics and Transaction Lists	19
4.9 Persistence, JDBC and Interface Technologies	20

TABLE OF CONTENTS (CONTINUED)

CHAPTER 5: PROBLEM STATEMENT 21
5.1 Existing Manual Process 21
5.2 Formal Problem Statement 22
5.3 Stakeholders and Their Needs 23
5.4 Functional Requirements 24
5.5 Non-Functional Requirements, Constraints and Assumptions 25
CHAPTER 6: DETAILED DESCRIPTION OF THE PROJECT 26
6.1 Proposed System and Layered Architecture 26
6.2 Users and Use-Case View 27
6.3 Module Description and Responsibilities 28
6.4 Account Opening and Validation Workflow 29
6.5 Deposit and Withdrawal Calculations 30
6.6 Withdrawal Decision Workflow 31
6.7 Balance Enquiry and Transaction History 32
6.8 Data Design and Entity Relationships 33
6.9 Class Design and Object Responsibilities 34
6.10 Conceptual Dashboard Layout 35
6.11 Conceptual Transaction Form and Feedback 36
6.12 Illustrative Dataset and Numerical Summary 37
6.13 Transaction-Volume Chart and Interpretation 38
6.14 Transaction-Mix and Balance-Trend Charts 39
6.15 Conceptual Testing, Security and Limitations 40
CHAPTER 7: CONCLUSION AND FUTURE SCOPE 41
7.1 Conclusion and Summary of Learning 41
7.2 Limitations and Future Scope 42
CHAPTER 8: REFERENCES 43
8.1 References and Source Notes 43

LIST OF TABLES

Table 1.1: Common types of bank accounts 2
Table 1.2: Organisation of the main report 4
Table 2.1: Objectives and the report evidence that supports them 6
Table 3.1: Training organisation and programme details from the supplied documents 8
Table 3.2: Four learning phases used to organise this report 9
Table 4.1: Access modifiers in simple terms 14
Table 4.2: Common Java data types and their uses 16
Table 4.3: Examples of validation and exception-related situations 18
Table 5.1: Stakeholders and their information needs 23
Table 5.2: Proposed functional requirements 24
Table 5.3: Non-functional requirements and assumptions 25
Table 6.1: Modules and their responsibilities 28
Table 6.2: Worked deposit and withdrawal calculations 30
Table 6.3: Illustrative transaction ledger for sample account A101 32
Table 6.4: Synthetic monthly activity dataset used in the charts 37
Table 6.5: Conceptual validation and security review plan 40
Table 7.1: Possible future extensions beyond the present theory report 42

LIST OF FIGURES

Figure 3.1: Connection between Java Programming training and the conceptual banking case study.	10
Figure 4.1: A simplified view of Java source code, compilation and execution.	12
Figure 4.2: One conceptual class and several independent account objects.	13
Figure 4.3: A conceptual inheritance relationship between general and specialised account types.	15
Figure 4.4: Conceptual relationship between Java logic, JDBC, a driver and stored records.	20
Figure 5.1: Simplified manual account-request process used as the problem context.	21
Figure 6.1: Conceptual layered architecture of the proposed account-management system.	26
Figure 6.2: Use-case view showing the main user goals within the conceptual system boundary.	27
Figure 6.3: Conceptual account-opening workflow with a correction path for invalid details.	29
Figure 6.4: Withdrawal workflow with validation, available-balance checking and a no-change rejection path.	31
Figure 6.5: Conceptual entity relationship design with primary and foreign key relationships.	33
Figure 6.6: Conceptual class diagram showing proposed fields, methods and responsibilities.	34
Figure 6.7: Illustrative dashboard layout; no running application or live account is represented.	35
Figure 6.8: Illustrative transaction form with validation and cancellation choices.	36
Figure 6.9: Monthly activity counts from the report's synthetic dataset; not measured system results.	38
Figure 6.10: Proportions of deposits, withdrawals and enquiries in the synthetic activity dataset.	39
Figure 6.11: Balance trend calculated from the fictional A101 ledger in Table 6.3.	39

ABBREVIATIONS AND NOTATION

Term	Meaning
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
ATM	Automated Teller Machine
BigDecimal	Java class for exact decimal values with controllable precision
CPU	Central Processing Unit
FK	Foreign Key
GUI	Graphical User Interface
INR	Indian Rupee; currency label used in the examples
JDBC	Java Database Connectivity
JDK	Java Development Kit
JRE	Java Runtime Environment
JVM	Java Virtual Machine
NDA	Non-Disclosure Agreement
OOP	Object-Oriented Programming
PK	Primary Key
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface

Reading note: Conceptual means proposed for explanation, not implemented. Illustrative or synthetic data are fictional learning examples. Amounts are shown in INR; no rupee-symbol font substitution is required.

CHAPTER 1 INTRODUCTION

1.1 Background of the Study

Banking is an important part of everyday economic life. People use bank accounts to keep money, receive income, make payments and save for future needs. Even a small branch or local banking service must maintain accurate information about account holders, available balances and previous transactions. A mistake in one entry can affect later calculations and reduce a customer's confidence in the service.

In a manual system, a clerk may write account details in a register, check an old balance and calculate a new balance after every request. This approach is understandable, but it becomes repetitive as the number of accounts increases. The same information may be written in several places. Searching for an old entry can take time, and a missed digit can produce an incorrect result.

A computer-based account management system organises these activities through a defined process. It accepts information, checks rules, updates records and presents useful results. The important idea is not simply replacing paper with a screen. The system must also make the rules consistent. For example, a withdrawal should not reduce an account below its permitted balance, and every successful financial transaction should have a traceable record.

This report studies that idea through a simple project titled Bank Account Management System Using Java. Java is suitable for the discussion because it supports object-oriented programming, structured error handling and clear separation of responsibilities. An account can be represented as an object, while deposits and withdrawals can be explained as operations performed according to rules [1].

The project is a conceptual academic case study connected with the student's one-month Java Programming training at EWB Courses. It explains the technology and demonstrates its application through diagrams, tables, interface layouts and manually calculated examples. It does not contain a working banking application, a connected database or real customer information.

The word system in this report therefore means a proposed organisation of information and activities. The diagrams show how a future application could be structured. The charts use a small synthetic dataset prepared for learning purposes. This approach keeps the project simple while still covering the problem, technology, design decisions and expected behaviour in a complete manner.

1.2 Bank Accounts and Basic Operations

A bank account is a record of a financial relationship between a customer and a banking institution. The record normally contains an identifier, the holder's details and information about permitted transactions. Different account products have different rules. This report concentrates on the common operations that help explain information handling without introducing complex banking products.

Table 1.1: Common types of bank accounts

Account type	Main purpose	Simple academic interpretation
Savings account	Encourages regular saving.	An account record with deposits, withdrawals and balance enquiry.
Current account	Supports frequent business transactions.	A separate account category with its own rules.
Fixed deposit	Keeps money deposited for a chosen term.	A time-based product; not implemented in this case study.
Recurring deposit	Supports regular deposits over a period.	Another specialised product outside the basic scope.

The first basic operation is account opening. A new record should contain enough information to identify the account and its holder. The account number must be unique so that two customers do not accidentally share the same financial record. In a real service, identity verification and regulatory checks are also necessary; those activities are outside this academic design.

The second operation is a deposit. A valid deposit increases the available balance. If an account contains INR 10,000 and a deposit of INR 2,000 is accepted, the new balance is INR 12,000. The system should also record what happened, when it happened and which account was affected.

The third operation is a withdrawal. A withdrawal decreases the available balance, but only when the request satisfies the account's rules. A request for more than the permitted amount should be rejected rather than creating an unexplained negative balance. This report assumes that no overdraft facility exists.

Balance enquiry and transaction history are information-retrieval operations. They should not change the financial balance. An enquiry reads the current state, while a history view explains how that state was reached. Keeping reading operations separate from balance-changing operations makes the design easier to understand and review.

1.3 Need for a Simple Conceptual Study

A beginner does not need a large banking platform to understand the relationship between a real-world problem and a software design. A small, carefully explained case study can show how information enters a process, how rules are checked and how records should change. This is the approach followed in the present report.

The first need is clarity. Banking terms can become complicated when interest, loans, investments, payment networks and regulatory reporting are introduced together. This study avoids those additional subjects and focuses on account details, deposits, withdrawals, balance enquiries and history. The limited scope makes it possible to explain each operation from beginning to end.

The second need is correctness. A visually attractive screen has little value if its underlying calculation is wrong. The report therefore gives attention to simple arithmetic, validation rules and the order of steps. A failed request should leave the account unchanged. A successful request should update the balance and create a matching transaction entry. These are design expectations, not claims based on software testing.

The third need is a connection with the training technology. Java Programming provides the vocabulary for discussing classes, objects, methods, data types and exceptions. The banking example gives those terms a practical meaning. A balance becomes a field that needs protection; a deposit becomes an operation with a rule; an incorrect amount becomes a situation that must be handled clearly.

The fourth need is honest documentation. A technology-based training report can demonstrate learning through a relevant problem and an appropriate example without disclosing company information. The supplied departmental instructions allow the report's organisation to reflect the nature of the training [13]. This report uses that flexibility by replacing live-application screenshots with labelled conceptual layouts.

Finally, the study provides a foundation for later academic work. A student who understands the account rules, data relationships and validation cases can later build a small supervised prototype. That future work would require separate implementation and testing. It is not presented as already completed here.

1.4 Scope and Organisation of the Report

The scope of this report is the explanation and conceptual design of a basic Bank Account Management System using Java. The study covers common account operations, suitable programming concepts, a manual-process problem, proposed requirements, workflows, data relationships and illustrative analysis.

The functional boundary is small: account opening, lookup, deposits, withdrawals, balance enquiries and transaction history. Closure is mentioned as a controlled state change. Transfers, loans, interest, ATM integration, payment gateways and real banking networks are excluded.

The technical boundary is also clear. Java explains the design. Database concepts and JDBC are possible supporting technologies, but no database is connected. Interface diagrams are static illustrations, not application screenshots, and no performance measurement is claimed.

The data boundary is important. Names, account numbers, transactions and chart values are fictional examples. They do not represent EWB Courses, any bank or any real customer. The supplied certificate is separate from this dataset.

Table 1.2: Organisation of the main report

Chapter	Title	Purpose
1	Introduction	Introduces banking, the topic and the report scope.
2	Objective of the Project	States the learning goals and expected outcomes.
3	Overview of the Company and Training	Describes the organisation and training context.
4	Overview of Technology	Explains Java and supporting concepts in simple language.
5	Problem Statement	Defines the manual-process problem and requirements.
6	Detailed Description of the Project	Presents workflows, diagrams, layouts and sample analysis.
7	Conclusion and Future Scope	Summarises learning and possible extensions.
8	References	Lists the sources and documents used.

The report moves from background and learning goals to the training context, technology and defined requirements. Chapter 6 explains the design through diagrams and examples. The final chapters summarise learning, identify limitations and list references.

This organisation follows the university structure while adapting the project description to a theory-only case study. Every item serves a clear purpose: explaining the problem, connecting it with Java, or showing how the design addresses the requirements.

CHAPTER 2 OBJECTIVE OF THE PROJECT

2.1 Main Objective

The main objective of this project is to explain how Java programming concepts can be applied to the conceptual design of a simple Bank Account Management System. The project uses a familiar banking problem to connect programming theory with a practical information-processing situation.

The study is not intended to produce a deployable financial product. Its purpose is to show that the student can identify a problem, describe its information needs, organise a solution and explain the result using suitable technical language. The proposed system is therefore treated as a learning example rather than a completed commercial service.

The first part of the objective is to understand the domain. A bank account has a current state, an owner and a history of operations. Deposits and withdrawals change that state, while enquiries only read it. Recognising this difference helps prevent unclear designs in which every button or method is allowed to modify important information.

The second part is to connect the domain with object-oriented programming. Java classes can represent account and transaction concepts. Objects can hold individual values, and methods can describe permitted operations. Encapsulation helps explain why a balance should not be changed freely from every part of a program [2].

The third part is to express rules before implementation. Requirements should state what counts as a valid amount, how an unknown account is handled and what happens when a withdrawal exceeds the available balance. Clear rules reduce ambiguity and provide a basis for future testing.

The fourth part is to communicate the design. Flowcharts, use cases, data diagrams and tables make the proposed process visible without requiring executable code. They also allow a reader to check whether the sequence of activities makes sense.

The final part is to interpret simple information. A manually prepared ledger shows how balances change. A synthetic activity dataset shows how tables can be converted into charts. These examples develop basic analytical skills while remaining clearly separate from real banking evidence.

Success in this report means a clear, consistent and honest explanation. It does not mean claiming a working model, real users, measured speed improvements or production-level security.

2.2 Specific Objectives and Supporting Evidence

The main objective can be divided into smaller objectives that are easier to study and evaluate. Each specific objective is connected with a visible part of the report. This makes the relationship between the learning goal and its supporting explanation clear.

Table 2.1: Objectives and the report evidence that supports them

Objective	Report evidence	Learning demonstrated
Understand basic account operations.	Sections 1.2 and 6.3 6.7.	Domain understanding.
Apply object-oriented thinking.	Sections 4.2 4.4 and 6.9.	Classes, objects and responsibilities.
Represent a process clearly.	Flowcharts and use-case diagram.	Logical sequencing.
Organise account information.	Entity relationship and class diagrams.	Basic data modelling.
Explain validation and safe behaviour.	Requirements and test-plan tables.	Attention to invalid inputs.
Interpret simple numerical information.	Sample ledger and charts in Chapter 6.	Tables, calculations and graphs.

The objectives follow a sensible order. Domain understanding comes first because a software design should reflect the problem it is intended to address. Programming concepts come next because they provide the tools for organising the solution. Process and data diagrams then connect those tools with the proposed account operations.

Validation is included as a separate objective because correct behaviour is not limited to successful examples. A system must also explain what happens when information is missing, an amount is invalid or an account cannot be found. The report treats these cases as part of the design rather than as occasional interruptions.

The communication objective is important for an internship report. A reader should be able to follow the project without running software. Tables identify requirements and responsibilities. Flowcharts show decisions. Interface layouts illustrate how information could be arranged. Captions and nearby explanations state what each visual element means.

The numerical objective is limited to basic interpretation. The report calculates closing balances, totals and proportions from deliberately simple data. It does not use those calculations to claim that a bank became more efficient. The charts are learning aids, not an evaluation of an implemented system.

Together, these objectives provide a balanced study of technology and its application. They also keep the project within a manageable size for a student completing a short Java Programming internship.

2.3 Expected Learning Outcomes and Boundaries

At the end of preparing this case study, the student should be able to explain the purpose of a basic account management system in straightforward language. The expected outcome is understanding, not the ability to operate a real banking platform.

The first learning outcome is the ability to identify entities and operations. Customer, account and transaction are different information concepts. Deposit and withdrawal are operations that change a balance; enquiry is an operation that reads information. Separating these ideas reduces confusion when discussing classes and database records.

The second outcome is the ability to connect Java concepts with a problem. A class can describe common structure, an object can represent one account, and a method can define a controlled operation. Private fields, validation and exceptions can be explained through examples that have an obvious purpose.

The third outcome is the ability to describe a process step by step. A withdrawal should not begin by subtracting money and only later check whether the request was valid. The correct conceptual order is to identify the account, check its status and amount, apply the rule and then record the result. Flowcharts make this order visible.

The fourth outcome is basic data-modelling awareness. An account should have a unique identifier. A transaction should refer to the correct account. A history view should not mix records from different accounts. These ideas can be discussed before selecting a particular database product.

The fifth outcome is responsible interpretation of examples. A sample ledger can demonstrate arithmetic, but it cannot prove reliability. A mock interface can demonstrate layout, but it cannot prove usability. A proposed access rule can describe security, but it cannot prove that an application is secure.

The boundaries support that honesty. No executable banking model, live dataset, company source code, NDA-protected material or real transaction evidence is included. No company NDA is asserted. The report follows the technology-based training route described in the departmental notice.

The expected final product is therefore a complete academic explanation: a defined problem, relevant Java theory, a proposed design, illustrative application and a clear statement of what remains to be implemented and tested.

CHAPTER 3 OVERVIEW OF THE COMPANY AND TRAINING

3.1 Profile of the Training Organisation

The student's supplied documents identify the training organisation as EWB EDUTECH PRIVATE LIMITED, presented publicly in the documents as EWB Courses. The programme named in the offer letter is Java Programming Internship for Beginners Learn, Build and Get Certified. The stated role is Java Programming Intern, and the mode is Virtual / Pan India.

Table 3.1: Training organisation and programme details from the supplied documents

Item	Documented detail
Organisation	EWB EDUTECH PRIVATE LIMITED, also shown as EWB Courses.
Programme	Java Programming Internship for Beginners Learn, Build and Get Certified.
Role and mode	Java Programming Intern; Virtual / Pan India.
Duration and stipend	One month; unpaid. The offer states that no programme fee is charged.
Offer reference	EWB-Java-Programming-2026-9111; offer dated 15 June 2026.
Certificate reference	EWB/INT/2026/9765; issued on 16 July 2026.
Contact shown in offer	support@elevatewithb.in; www.ewbcourses.com.

The organisation's documents describe a learning programme rather than an employment relationship. The offer states that the internship is unpaid and that no registration, training, security or placement fee is charged. It also states that the programme does not create an employer employee relationship or include employment benefits.

This report uses the supplied offer letter and completion certificate as the source for organisation and programme details. It does not independently claim information about the organisation's revenue, number of employees, clients or market position. Keeping the profile limited to documented facts avoids introducing unsupported company information.

The training is relevant to the project because Java Programming is the central subject of the certificate. The banking example is an academic case selected for this report; it should not be interpreted as a claim that EWB operates a bank or supplied real banking data.

The certificate's registration and recognition markings belong to the original document and are reproduced only in the colour certificate page. The technical discussion does not rely on them.

The student name used in this report follows the completion certificate and the issued-to field of the offer letter. The banking case study remains separate from these personal training documents. No claim is made that the training organisation provided, endorsed or operated the proposed banking design.

3.2 Training Duration and Learning Themes

The offer letter describes a one-month internship, and the completion certificate confirms successful completion of a one-month internship in Java Programming. The offer is dated 15 June 2026, while the certificate was issued on 16 July 2026. These dates are document dates; the offer itself lists the commencement as to be announced and the conclusion as one month from the start.

The report therefore describes the training as a one-month programme completed in the June July 2026 period. It does not invent an exact daily attendance record or a session calendar that was not supplied. This distinction keeps the academic description consistent with the available evidence.

The offer lists participation expectations that include attending scheduled sessions, completing weekly assessments and activities, completing a final mini project and following programme instructions. For this report, the learning is organised into four broad phases. The phases are a convenient way to present the subject matter, not a reproduction of an official day-by-day timetable.

Table 3.2: Four learning phases used to organise this report

Phase	Learning theme	How the theme is represented
Week 1	Java basics and programming vocabulary.	Chapter 4 explains the platform, variables and control flow.
Week 2	Classes, objects and object-oriented principles.	Class/object examples and inheritance discussion.
Week 3	Validation, collections and data organisation.	Error-handling discussion, data design and ledgers.
Week 4	Case-study organisation and review.	Banking workflows, illustrative charts and limitations.

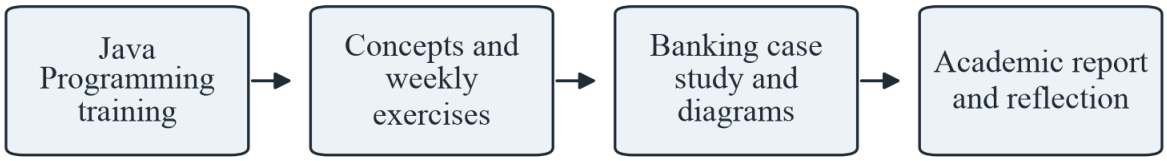
The first phase introduces the vocabulary needed to read and discuss Java programs. The second phase explains how information and behaviour can be grouped into classes and objects. The third phase considers what happens when inputs are incorrect and how related records can be organised. The fourth phase brings the concepts together in a small case study.

This progression is suitable for a beginner because each stage depends on the previous one. A student should understand variables and decisions before discussing validation, and should understand classes before interpreting a class diagram.

The final output submitted with this academic report is a theory-based project document. The offer's mini-project requirement is a programme instruction; this report does not claim that a working banking application was delivered to the company. It demonstrates the learned technology through a relevant problem, worked examples and conceptual design, as permitted for technology-based training.

3.3 Technology-Based Training and the Case-Study Route

The departmental notice allows students who completed technology-based training to explain the technologies learned, select a relevant problem or case study and demonstrate practical application through an appropriate example. This report follows that route because the supplied certificate identifies a Java Programming internship rather than a placement in a banking organisation.



Learning is represented through explanation, not a deployed application.

Figure 3.1: Connection between Java Programming training and the conceptual banking case study.

The learning path begins with the training subject. Java provides the programming concepts. Those concepts are then used to describe an account-management problem. Finally, the explanation is organised into a report that a reader can evaluate without access to a company system.

A case study is useful because it gives abstract terms a setting. Encapsulation becomes easier to understand when a balance should only change through approved operations. Validation becomes meaningful when a negative deposit must be rejected. Association becomes clearer when a transaction belongs to one account.

The report shows application without unsupported claims: requirements describe behaviour, flowcharts describe decisions, data diagrams describe relationships, calculations show arithmetic, and layouts show possible information placement.

The notice also refers to NDA cases. No NDA document has been supplied, and this report does not state that one was signed. The technology-based training route is the relevant basis here. Any separately signed NDA would require proof through the institution's submission process.

No confidential company code, credentials, private recordings or restricted business data are included. The organisation profile uses the student's own offer and certificate. All banking records and numerical datasets are fictional.

This approach explains the learning, applies the technology to a suitable problem and separates academic illustration from professional or confidential evidence.

3.4 Professional Practices and Documentation Habits

Technical learning also includes habits that make work understandable and reviewable. A program or design should not depend on memory alone. Names, rules, assumptions and sources should be recorded so that another reader can follow the reasoning.

The first habit is clear naming. In the banking example, accountNumber, holderName and balance communicate their purpose more directly than vague names such as x1 or data2. Consistent names should be used across requirements, diagrams and explanations. A field should not be called customerCode in one section and accountNumber in another without a reason.

The second habit is separating responsibilities. Input checking, account rules and presentation should not be mixed into one unclear description. When each part has a defined job, a change in the screen layout need not change the deposit rule. This idea later appears in the proposed layered architecture.

The third habit is recording assumptions. The case study assumes a fictional bank context, no overdraft, basic account operations and synthetic data. Those assumptions explain why certain situations are outside the scope. Without them, a reader might expect interest calculation or real payment processing and incorrectly judge the design incomplete.

The fourth habit is reviewing edge cases. A deposit example should include not only a valid positive amount but also zero, a negative value and invalid text. A withdrawal example should include an amount greater than the available balance. Considering these cases develops discipline even when no code is executed.

The fifth habit is careful documentation. Diagrams need captions, tables need headings and sample data needs an explicit label. Sources should be acknowledged rather than presented as original material. The certificate should be reproduced without alteration, and personal or organisational details should match the supporting documents.

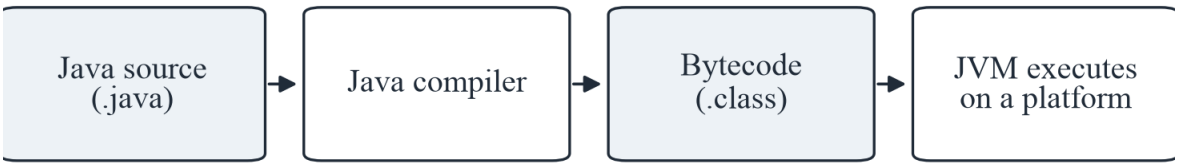
The sixth habit is honest reporting. A design is not a tested product, and a chart made from invented learning data is not measured business performance. Stating that difference clearly is part of professional communication.

These habits connect the Java training with the final report. They help produce work that is simple to read, consistent in its rules and transparent about its evidence.

CHAPTER 4 OVERVIEW OF TECHNOLOGY

4.1 Java and the Java Platform

Java is a general-purpose programming language and computing platform. It is widely used for teaching because it combines a structured language with strong support for object-oriented design. Java applications cover many areas, but this report uses only the concepts needed for a small account-management example [1].



Write once, compile anywhere; a compatible JVM is still required.

Figure 4.1: A simplified view of Java source code, compilation and execution.

A programmer writes instructions in a source file. A compiler checks the source and produces bytecode. The Java Virtual Machine, or JVM, executes that bytecode on a particular platform. This arrangement explains the common phrase write once, run anywhere : the same bytecode can be used on different systems that provide a compatible Java environment.

The Java Development Kit, or JDK, contains tools needed to develop Java programs, including the compiler. The Java Runtime Environment is the environment needed to run Java applications. In modern Java distributions, the runtime is provided through the platform's runtime arrangements rather than requiring the beginner to treat every term as a separate product.

For this project, the important point is the separation between source instructions, compiled form and execution environment. It shows why Java programs are not tied directly to one computer's machine instructions.

Java also provides a large standard library. Collections help organise groups of values, exception types help describe failures, and decimal-number classes support careful numerical work. A student can therefore focus on the problem's rules instead of inventing every supporting structure.

The report discusses Java conceptually. It does not install a JDK, compile a banking application or measure execution speed. Understanding the platform provides the foundation for the later class, validation and data-design discussions.

4.2 Classes, Objects and Methods

A class is a description of structure and behaviour. An object is an individual instance of that description. In the proposed banking study, a BankAccount class can describe the information and operations common to accounts, while separate objects can represent sample accounts A101 and A102.

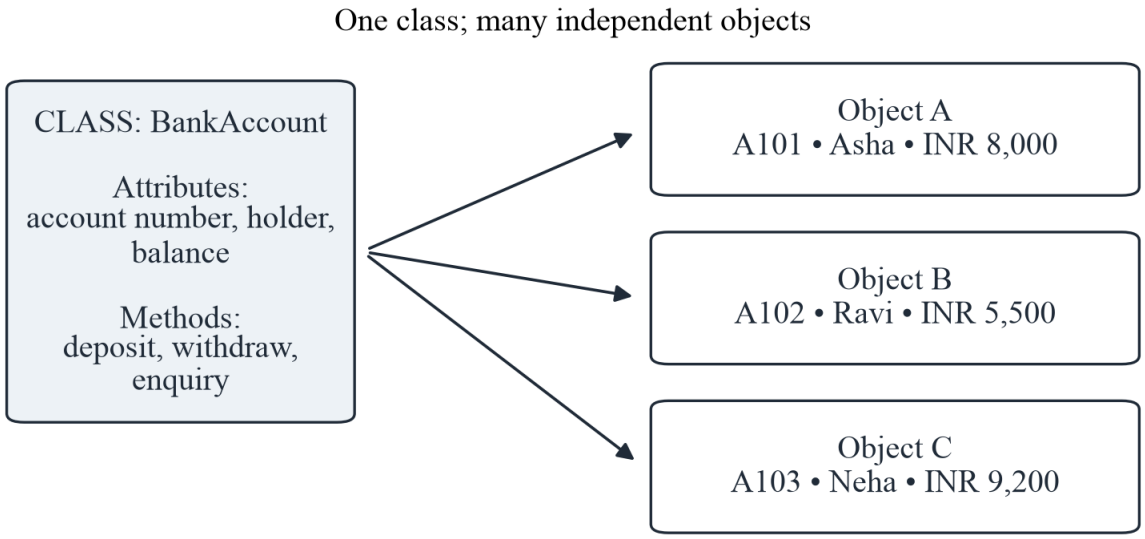


Figure 4.2: One conceptual class and several independent account objects.

The structure is represented by fields. An account may have an account number, holder name, type, status and balance. Each object maintains its own values. A deposit into A101 should not change A102's balance. This simple observation explains why state must belong to the correct object.

Behaviour is represented by methods. A deposit method describes how a valid amount affects the account. A withdrawal method describes the checks and the permitted reduction. An enquiry method returns information without changing the financial state. Method names should communicate these responsibilities clearly.

A constructor is a special member used when an object is created. Conceptually, it can establish the initial account details and opening balance. The design should not allow an object to begin with an unclear identity or an undefined financial state.

Classes also help organise change. If the wording of a validation message changes, that change should belong to the validation or presentation responsibility rather than being repeated throughout every account operation.

The diagram is a learning illustration, not generated from compiled code. Its purpose is to show the relationship between a general definition and individual examples. This is one of the most important connections between Java theory and the banking case study.

4.3 Encapsulation, Abstraction and Access Control

Encapsulation means keeping related information and the operations that control it together. In a bank-account class, the balance should not be treated as an ordinary public value that any other part of a program can overwrite. Controlled operations make the intended rules visible [2].

For example, a deposit should accept a valid positive amount and then update the balance. A withdrawal should first check the account and available funds. If outside code could assign any value directly to the balance, these checks could be bypassed. Private fields and carefully designed public methods help explain the protection.

Table 4.1: Access modifiers in simple terms

Modifier	Meaning	Banking example
private	Accessible inside the declaring class.	Keep the balance field inside BankAccount.
public	Available to other classes where the class itself is accessible.	Expose a carefully designed enquiry method.
protected	Available within the package and through inheritance rules.	Useful for selected behaviour shared with subclasses.
Package-private	Available within the same package when no modifier is written.	Group closely related helper classes.

Abstraction is closely related but focuses on essential behaviour. A user of an account object needs to know that a withdrawal can be requested and that a result will be returned. The user does not need to understand every internal comparison or record-building step before using the operation.

A simple interface can express this idea. It might describe operations such as deposit, withdrawal and balance enquiry without specifying all internal details. A class can then provide the behaviour. This separates what operation is available from how the operation is performed.

Access control is a language mechanism; it is not complete banking security. A private Java field does not authenticate a person, encrypt a database or protect a network connection. Those are separate concerns. The report makes this distinction so that a beginner does not confuse object-oriented protection with production security.

In the proposed design, encapsulation supports consistent account rules, abstraction keeps the discussion manageable, and access modifiers express which members are intended for internal or external use. Together, they make the conceptual model easier to reason about and safer to extend in future supervised work.

4.4 Inheritance and Polymorphism

Inheritance allows a class to build on the structure and behaviour of another class. It is useful when several concepts share a genuine is a relationship. A savings account and a current account can both be treated as kinds of bank account while retaining some product-specific behaviour.

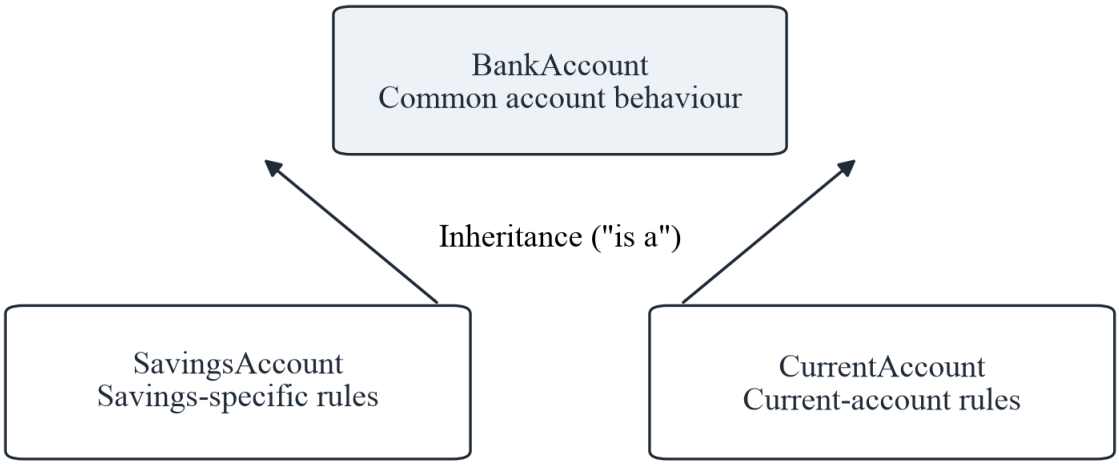


Figure 4.3: A conceptual inheritance relationship between general and specialised account types.

The general BankAccount class can contain common information and operations. Specialised classes can add or adjust behaviour where the rules genuinely differ. This avoids repeating every common field in several unrelated descriptions.

Polymorphism means that the same general operation can be handled through different specific forms. Code can refer to a general account while the actual object provides the appropriate specialised behaviour. In a small academic example, this helps explain how common processing can work with more than one account type.

Inheritance should not be used merely because it is available. A transaction is not a kind of account; it is an event associated with an account. That relationship should be modelled as an association, not inheritance. Choosing the correct relationship is more important than creating a large class hierarchy.

The present case study keeps inheritance limited. Most examples use an ordinary account so that deposits, withdrawals and validation remain easy to follow. Savings and current accounts appear as possible specialisations, but detailed interest, fees and overdraft rules are outside the scope.

This restrained use of object-oriented concepts keeps the project suitable for a beginner. It demonstrates the principle without allowing unnecessary class structures to hide the basic banking process.

4.5 Variables, Data Types and Money Values

A variable is a named place for a value used by a program. A data type describes what kind of value can be stored and what operations make sense. Choosing suitable types helps prevent confusion between a count, a label, a status and a monetary amount.

Table 4.2: Common Java data types and their uses

Type	Stores	Example use in this study
int / long	Whole numbers; long has a wider range.	Counts of entries or an internal identifier.
boolean	true or false.	Whether an account is active.
double	Approximate floating-point numbers.	Charts or general numerical examples, not exact money.
BigDecimal	Decimal values with controllable precision.	Preferred design choice for monetary amounts.
String	A sequence of characters.	Account number, holder name and remark.
LocalDateTime	Date and time without a time-zone offset.	A transaction timestamp in a defined system context.

Account numbers should normally be stored as text rather than numbers used for arithmetic. An account identifier may contain leading zeros or letters, and adding two account numbers has no useful meaning. A holder's name is also text. An active-or-closed status can be represented by a boolean in a very small example or by a clearly defined set of states in a larger design.

Monetary values need special care. Binary floating-point types such as double are useful for many scientific and graphical calculations, but they cannot represent every decimal fraction exactly. A design that repeatedly adds and subtracts money should not depend on approximate representation.

Java's BigDecimal class supports decimal values with explicit precision and rounding rules [3]. In a future implementation, amounts can be created from exact textual values and compared using suitable decimal methods. The report recommends this approach conceptually without presenting executable financial code.

The examples use INR values with two decimal places because that makes the ledger easy to read. No rounding rule is needed for the whole-number sample amounts, but a real implementation would need a documented policy for every calculation that can produce additional decimal places.

Type choice is therefore part of correctness. It is not only a syntax decision. The right type communicates the meaning of the information and reduces avoidable errors before the business rules are even considered.

4.6 Control Flow and Method Organisation

Control flow is the order in which instructions are considered. A simple program may follow instructions one after another, but most useful processes also make decisions and repeat activities. Java supports these patterns through conditional statements, loops and method calls.

Sequential flow appears in a deposit explanation. The process receives an amount, checks it, calculates a proposed balance and prepares a record. The order matters because the calculation should not become a committed change before the checks succeed.

Selection appears when the process must choose between alternatives. If an account exists, the process continues; otherwise, it returns an account-not-found result. If a withdrawal is within the permitted balance, it may proceed; otherwise, it is rejected. These decisions are represented by branches in the flowcharts.

Iteration appears when a history view examines several transaction entries. The same display or totaling step can be repeated for each relevant record. A loop should operate on the correct collection and have a clear stopping condition. In this report, the idea is explained through a ledger rather than through code.

Methods help divide a process into named responsibilities. A small design might separate amount validation, account lookup and transaction recording. Each method should have a clear input, output and purpose. A method that performs unrelated tasks becomes difficult to explain and test.

A return value communicates a result. For a balance enquiry, the result may be a decimal value. For a deposit or withdrawal, the result should indicate success or a defined failure reason and may include updated information. Simply printing a message is not enough to describe the operation's effect on the account.

Parameters allow information to enter a method. The account identifier and amount are examples. They should be checked rather than trusted automatically. Clear parameter meaning also makes documentation easier.

The project uses these control-flow ideas to describe predictable behaviour. The later workflows are not executable programs; they are structured explanations of the sequence a future Java implementation should follow.

4.7 Validation and Exception Handling

Validation is the process of checking information before it is used. Exception handling is a way to manage unusual conditions that occur during execution. The two ideas are related but not identical. Expected input problems should be checked clearly, while exceptions can represent failures that interrupt normal processing [4].

Table 4.3: Examples of validation and exception-related situations

Situation	Expected response	Concept used
Amount is zero or negative.	Reject the transaction without changing balance.	Input validation.
Account number is not found.	Show a clear account-not-found message.	Lookup failure handling.
Withdrawal exceeds balance.	Reject the request and retain the existing balance.	Business-rule check.
Text entered where a number is expected.	Explain the input problem before conversion fails further.	Parsing and exception awareness.
Stored record cannot be read.	Do not invent a balance; report the failure safely.	Controlled error handling.

A negative deposit is an invalid request, not a reason to damage the account record. The design should reject it with a useful message. An empty account number is another input problem. A missing database connection in a future implementation is a different kind of failure and may require exception handling and recovery behaviour.

Java provides exception-handling structures such as try, catch and finally. A try block contains operations that may fail. A catch block can respond to a matching exception. A finally block can perform necessary cleanup. The beginner should understand the purpose of these structures without using them to hide every possible problem.

Messages should be understandable without revealing sensitive internals. Enter an amount greater than zero is more useful to a customer than a raw stack trace. At the same time, a future application may keep appropriate technical diagnostic information in a protected log for authorised support staff.

The report distinguishes three outcomes: a valid request succeeds, an invalid request is rejected without changing financial state, and an unexpected failure is reported safely. The third outcome cannot be fully demonstrated without an implementation, so it remains a requirement for future testing.

Good error handling supports both correctness and trust. It prevents silent failure, avoids unexplained balance changes and gives the user a clear next step. These principles appear throughout the proposed account workflows.

4.8 Collections, Generics and Transaction Lists

A collection is an object that groups related values. Java's collections framework provides common structures such as lists, sets and maps. Choosing the right structure depends on how the information will be searched, ordered and protected from duplication [5].

A List maintains an ordered sequence and can contain repeated values. It is suitable for explaining a transaction history in which entries appear in a defined order. Two deposits may have the same amount, so the amount itself cannot be used as a unique identity.

A Set represents a collection that does not allow duplicate elements according to its rules. It can help explain a group of distinct account types or identifiers. The definition of equality must be clear when objects are placed in a set.

A Map associates keys with values. An account number can act as a key leading to an account object in a small in-memory example. A lookup should still handle the case where the key does not exist. A map is not a permanent database, and its contents would disappear when an ordinary process ends unless persistence is added separately.

Generics allow a collection to state the type of object it contains. A list of Transaction objects is clearer and safer than a list of unrelated values. The compiler can then help detect type mistakes before execution.

Iteration is used to examine entries one by one. A history display might read each transaction for the selected account and present its type, amount and timestamp. A summary might count deposits and withdrawals. The report's charts follow the same idea using a manually prepared table.

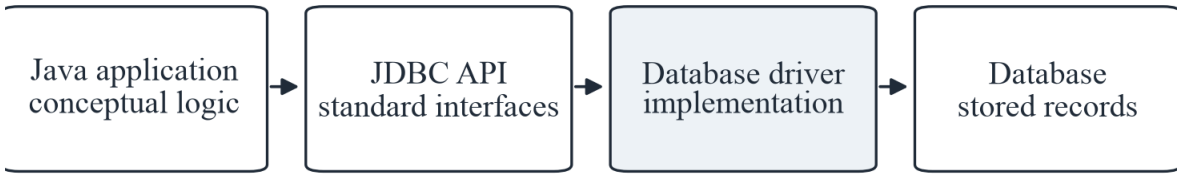
Sorting should use an explicit rule. Transaction history may be ordered by timestamp and then by a stable identifier when times match. Sorting by amount would answer a different question and should not be confused with chronological history.

Collections are useful for learning, but a real financial system needs durable storage and controlled concurrent access. The report therefore uses collections to explain organisation, not to claim that an in-memory list is sufficient for banking.

4.9 Persistence, JDBC and Interface Technologies

Persistence means keeping information beyond the lifetime of one running process. A banking design needs durable records so that balances and transaction history remain available after a restart. A relational database is one possible approach.

JDBC, or Java Database Connectivity, is a standard API through which Java code can interact with database systems. A suitable driver connects the standard API calls to a particular database product [6].



A possible persistence design; no database is deployed for this report.

Figure 4.4: Conceptual relationship between Java logic, JDBC, a driver and stored records.

In a future implementation, account and transaction tables could be created with keys and constraints. Prepared statements could carry values separately from the SQL structure. A balance update and its transaction entry would need to be committed together so that one cannot succeed while the other is lost.

For a desktop learning prototype, Java Swing is one possible interface toolkit. JavaFX is another. A web interface could also be built using separate server and browser technologies. The choice affects presentation but does not replace the underlying account rules.

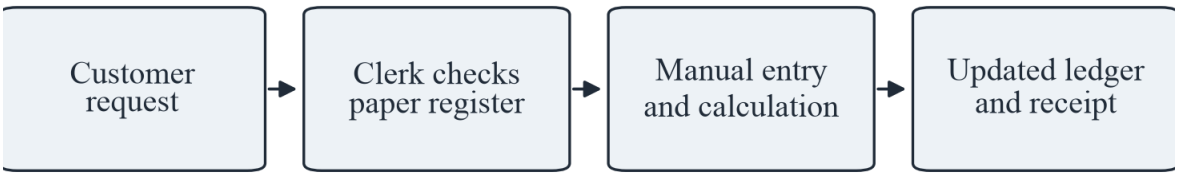
This report uses static interface layouts rather than selecting and running a toolkit. The layouts show where account details, actions and messages could appear. They are intentionally labelled as illustrations so that they are not mistaken for screenshots.

Development tools could include a Java IDE, a version-control system and documentation software. These tools support editing and review, but their presence alone does not prove correctness. The project's main evidence remains the written explanation, diagrams and worked examples.

CHAPTER 5 PROBLEM STATEMENT

5.1 Existing Manual Process

The problem situation is a fictional small banking counter that records basic account requests manually. A customer provides an account identifier and request. A clerk searches a register, checks the latest balance, performs a calculation and writes a new entry. The example is used to study information handling, not to describe an actual EWB client.



Repeated writing and checking can introduce delays and inconsistent records.

Figure 5.1: Simplified manual account-request process used as the problem context.

The process can work for a small number of accounts, but it requires repeated attention. The clerk must locate the correct record, read the latest entry and avoid copying mistakes. A customer may wait while previous pages are checked. If an entry is unclear, the next calculation may begin from an uncertain value.

Manual records can also be difficult to summarise. Counting deposits for a month may require reading many pages. Preparing a customer's history may involve finding entries spread across a register. These are plausible process difficulties, not measured results from a real branch.

The problem is not that paper is always wrong. A carefully maintained manual ledger can be accurate. The issue is that repetition, search and calculation depend heavily on individual effort. A defined computer-based process could make the rules more consistent and the information easier to retrieve.

The proposed study therefore focuses on four needs: reliable identification, consistent validation, traceable updates and clear retrieval. The following sections convert these needs into a precise problem statement and requirements. No claim is made that the present theory-only project has already removed delays or errors.

5.2 Formal Problem Statement

Problem statement: design and explain a simple Java-oriented conceptual system that organise basic bank-account information, applies consistent deposit and withdrawal rules, supports balance and history enquiries, and presents the process through clear diagrams and illustrative data, without using real banking information or claiming a working implementation.

The problem has an information component. Each account needs a stable identifier and a defined current state. Each financial transaction needs a relationship with exactly the intended account. The design must explain how these records differ and how they remain connected.

The problem also has a process component. A request should pass through identification, validation and decision steps before a balance changes. Invalid requests should produce an understandable result. The process must avoid partial updates in which a balance changes but its transaction record is missing.

A third component is presentation. Customers and clerks need understandable labels, amounts and messages. A screen should distinguish an action that changes information from an enquiry that only reads it. The conceptual layouts in Chapter 6 address this need without pretending to be a completed interface.

A fourth component is learning. The solution must connect the problem with Java concepts taught in a beginner programming context. Classes, objects, methods, encapsulation, data types, collections and exceptions should appear where they help explain the design. Technology should not be listed without a purpose.

The proposed answer is a layered conceptual design with defined account operations, validation rules, data relationships and sample analysis. Its success criteria are completeness, consistency and clarity. It should be possible to trace each main requirement to an explanation or diagram.

The statement deliberately excludes live integration, real customer verification, interest, transfers and production security. Those subjects would introduce additional rules and risks. A future implementation would also need actual testing against the requirements.

This bounded problem is appropriate for the report because it is specific enough to guide the design but small enough to explain fully without a working model.

5.3 Stakeholders and Their Needs

A stakeholder is a person or group affected by a system or interested in its information. Identifying stakeholders helps explain why a design needs particular features. The stakeholders below are roles in the fictional case study, not identified employees or customers of a real organisation.

Table 5.1: Stakeholders and their information needs

Stakeholder	Main need	Design consideration
Customer	Correct balance and understandable transaction information.	Clear enquiry and receipt views.
Bank clerk	A consistent way to record approved requests.	Simple forms and validation messages.
Authorised manager	Reliable summaries and traceable entries.	Unique identifiers and transaction history.
Training student	A small problem that connects Java concepts.	Limited scope and theory-based explanation.
Report evaluator	Clear evidence of learning.	Diagrams, tables, references and honest limitations.

The customer mainly needs accurate and understandable information. A balance should refer to the correct account, and a receipt should show the operation and resulting value. Messages should explain invalid input without unnecessary technical detail.

The clerk needs a consistent process for recording approved requests. Repeated information should not be entered in unrelated places. The form should make the account identifier, operation type and amount easy to distinguish.

An authorised manager may need summaries and traceable entries. This does not mean unrestricted access to every detail. Access should match responsibility, and financial changes should remain attributable in a future implementation.

The student needs a problem that connects programming concepts without becoming a full banking product. The evaluator needs clear evidence that the report covers the required structure and explains the technology honestly.

Considering these needs together prevents the design from becoming only a collection of screens. Each visual element should support a stakeholder's information task, and each rule should protect the integrity of the account information.

The report does not claim interviews, surveys or observations were conducted with these stakeholders. The needs are reasoned design assumptions for the academic example. Real requirements would require authorised discussion and validation.

5.4 Functional Requirements

Functional requirements describe what the proposed system should do. They provide a checklist for the conceptual design and a starting point for any later implementation. The requirements below are intentionally limited to basic account management.

Table 5.2: Proposed functional requirements

ID	Requirement	Acceptance condition in a future implementation
FR-01	Open an account.	Required details are checked and a unique account number is assigned.
FR-02	Find an account.	A valid identifier retrieves the correct account record.
FR-03	Record a deposit.	A valid positive amount increases the balance once.
FR-04	Record a withdrawal.	A valid request succeeds only when permitted by balance and status.
FR-05	Enquire about balance.	The displayed value matches the stored account state.
FR-06	Show transaction history.	Entries are associated with the correct account and shown in order.
FR-07	Validate inputs.	Invalid requests produce a message and no partial update.
FR-08	Close an account conceptually.	Closure follows stated rules and prevents later ordinary transactions.

The requirements use observable conditions wherever possible. Handle accounts properly is too vague to evaluate. A valid identifier retrieves the correct account record gives a clearer condition. Similarly, a deposit requirement should state both the balance effect and the need to avoid duplicate updates.

Each requirement should have both a normal case and at least one failure case. A valid deposit increases the balance; a negative deposit does not. A known account can be retrieved; an unknown identifier returns a clear result. This pairing makes the requirements more useful than a list of successful actions alone.

These are specifications, not test results. No software was run. Chapter 6 connects the requirements with workflows and a future review plan. Numbering improves traceability: FR-03 links to the deposit calculation, FR-04 to the withdrawal flowchart and FR-06 to the transaction ledger.

5.5 Non-Functional Requirements, Constraints and Assumptions

Non-functional requirements describe qualities expected from a system. They are important because a function can produce a result while still being difficult to use, unreliable or unsafe. In a theory report, these qualities are stated as design goals rather than measured achievements.

Table 5.3: Non-functional requirements and assumptions

Area	Requirement or assumption	Status in this report
Usability	Labels and messages should be easy to understand.	Shown through conceptual layouts.
Reliability	A failed request should not leave half-updated information.	Specified, not experimentally verified.
Data accuracy	Money should use exact decimal representation.	Explained through worked examples.
Security	Access should be restricted and sensitive data protected.	Design discussion only.
Maintainability	Responsibilities should be separated into clear modules.	Represented in the architecture.
Scope assumption	One fictional bank context and basic account operations.	No live bank or customer connection.

Usability is addressed through clear labels and predictable messages. Reliability is addressed through the rule that failed requests should not leave partial updates. Accuracy is supported by exact decimal representation and worked arithmetic. Maintainability is supported by separating responsibilities.

Security requires more than a private field or a login screen. A future system would need authentication, authorisation, protected storage, safe error handling and careful logging. This report discusses those ideas but does not claim to provide production-level protection.

The main constraint is the absence of a working model. Diagrams and examples can demonstrate reasoning, but they cannot establish response time, concurrency behaviour, accessibility or recovery after a real failure. Those would require implementation and testing.

The assumptions are equally important. The example uses one fictional banking context, basic account operations, no overdraft and no interest calculation. Amounts are expressed in INR. Account numbers are identifiers rather than arithmetic values. All customer names and transaction records are invented.

The report also assumes that the reader is interested in learning rather than operating a financial service. This keeps the explanations direct and avoids unnecessary product complexity.

Together, the requirements, constraints and assumptions define a realistic academic boundary. They make it possible to evaluate whether the report explains the chosen problem well without judging it as something it never claimed to be.

CHAPTER 6 DETAILED DESCRIPTION OF THE PROJECT

6.1 Proposed System and Layered Architecture

The proposed Bank Account Management System is a conceptual design for organising basic account information and operations. It is organised into presentation, business-rule and data responsibilities. The separation keeps the explanation clear and provides a sensible structure for a future Java prototype.

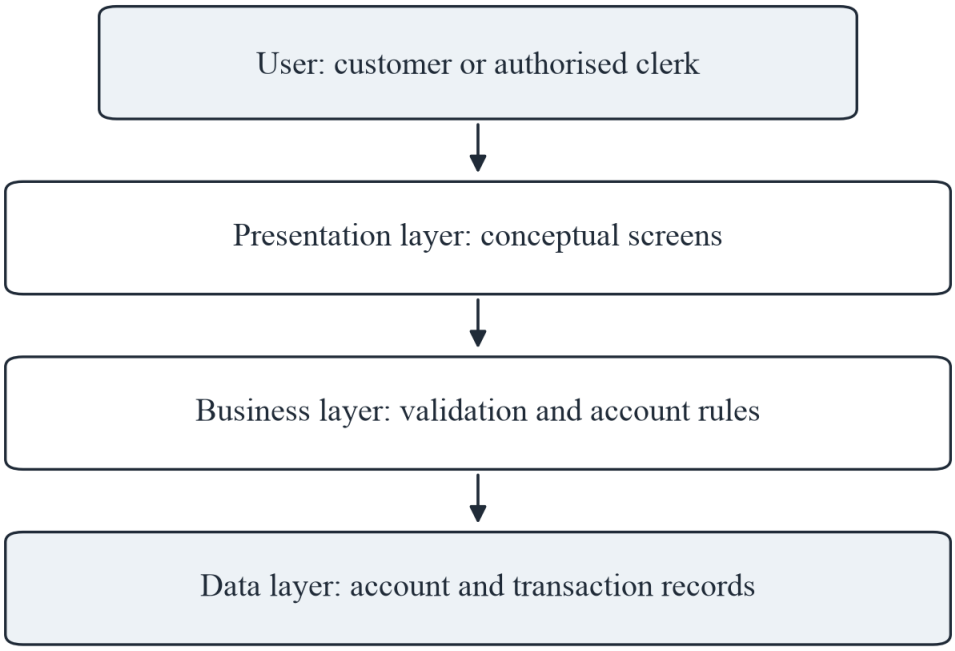


Figure 6.1: Conceptual layered architecture of the proposed account-management system.

The presentation layer represents the information shown to a customer or authorised clerk. It includes account forms, action choices, balance views and messages. Its responsibility is communication, not deciding whether a withdrawal is financially valid.

The business layer contains the account rules. It checks amounts, account status and available balance. It coordinates a successful balance change with a transaction record. This layer should return a defined result when a request is rejected.

The data layer represents stored account and transaction information. In a future implementation it could use a database accessed through JDBC. In the present report it is described through an entity relationship diagram and sample tables.

The layered explanation is technology-independent at the rule level but suitable for Java implementation. Classes and services can represent the business responsibilities, while interface components and database access remain separate.

No deployed components or network connections are implied by the diagram. It is a design view, not an infrastructure audit. Its value is that each requirement has a clear place in the proposed organisation.

6.2 Users and Use-Case View

The conceptual system has two main user roles. A customer mainly requests information and authorised transactions. An authorised clerk records approved requests and manages account information within defined permissions. Authentication and role assignment would need separate design in a working system.

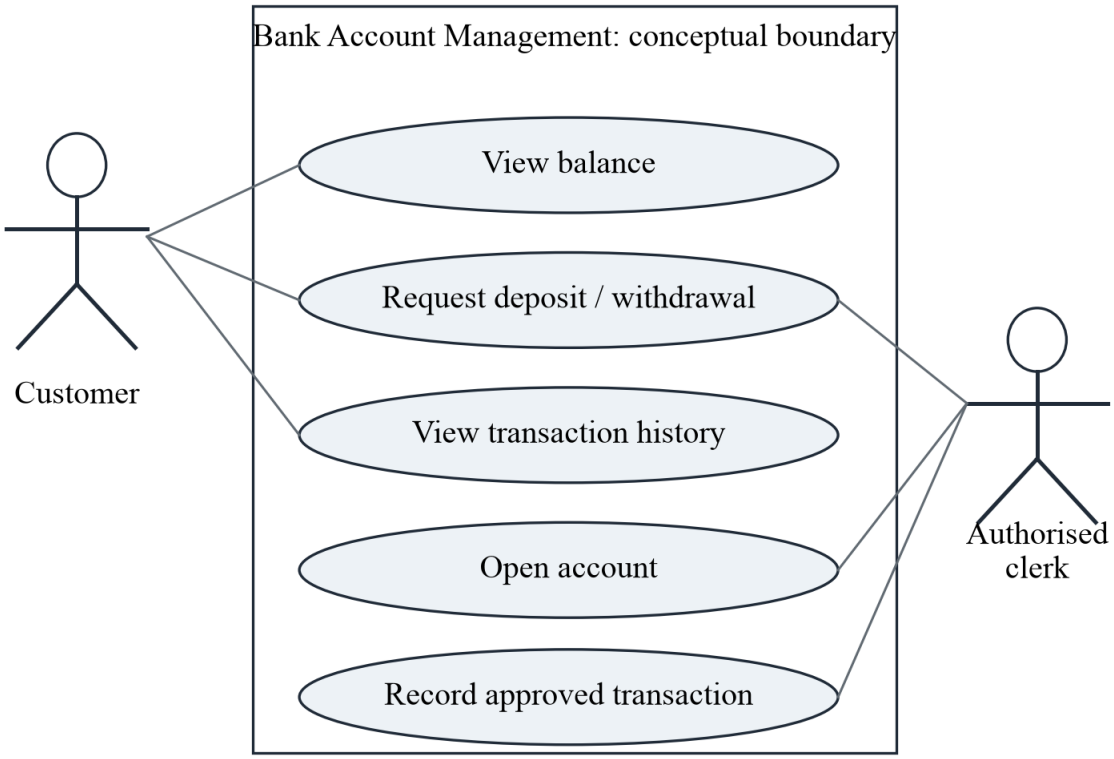


Figure 6.2: Use-case view showing the main user goals within the conceptual system boundary.

Balance enquiry is a read-only use case. Deposit and withdrawal requests lead to validation and may change the account state. Transaction history retrieves related entries. Account opening creates a new record, while approved transaction recording emphasises the clerk's responsibility.

The diagram groups goals rather than buttons. View balance is a user goal; a particular blue rectangle on a screen is only one possible way to support it. Keeping the discussion at the goal level prevents the interface layout from controlling the entire design.

Each use case needs a normal path and an alternative path. A withdrawal normally succeeds when the account and amount are valid. Its alternative path rejects an unknown account, inactive status or insufficient balance. These alternatives are part of the requirement, not optional additions.

The diagram does not show a real login service or permission engine. It identifies intended responsibilities so that later security design can decide who may perform each action. No unauthorised person should be able to view or change account information merely because a button is visible.

6.3 Module Description and Responsibilities

Modules divide the proposed system into understandable parts. A module should have a clear purpose and should not take responsibility for every activity. The following division supports the architecture and keeps the Java-oriented explanation suitable for a beginner.

Table 6.1: Modules and their responsibilities

Module	Responsibility	Important restriction
Account management	Create, locate and describe an account.	Account numbers must remain unique.
Transaction handling	Apply deposit and withdrawal rules.	Do not change balance after a failed check.
Validation	Check required fields, amount and account state.	Return understandable reasons for rejection.
Enquiry and history	Present balance and related entries.	Show only records for the selected account.
Data management	Define how records are stored and linked.	No actual database is deployed here.
Reports and charts	Summarise clearly labelled sample data.	Do not present examples as bank performance.

The account-management module deals with identity and state. It should distinguish an active account from a closed one and should prevent duplicate account numbers. It does not need to know how a screen is drawn.

The transaction-handling module applies financial operations. It depends on validation and account information. A successful operation should have one defined effect, while a rejected operation should have no financial effect.

Validation explains why a request cannot continue. Enquiry and history present information without changing balances. Data management defines how records would be stored. Reports and charts summarise data for learning or authorised review.

The boundaries between modules also make future testing easier. Amount checks can be reviewed separately from account lookup, and ledger arithmetic can be checked separately from interface appearance.

In this report, the modules are described through text, diagrams and examples. They are not compiled packages or deployed services. The module table should therefore be read as a responsibility plan for future implementation.

6.4 Account Opening and Validation Workflow

Account opening establishes a new account record. The conceptual workflow collects required details, checks their format, assigns a unique account number and prepares confirmation. Real identity verification and regulatory checks are outside the academic scope.

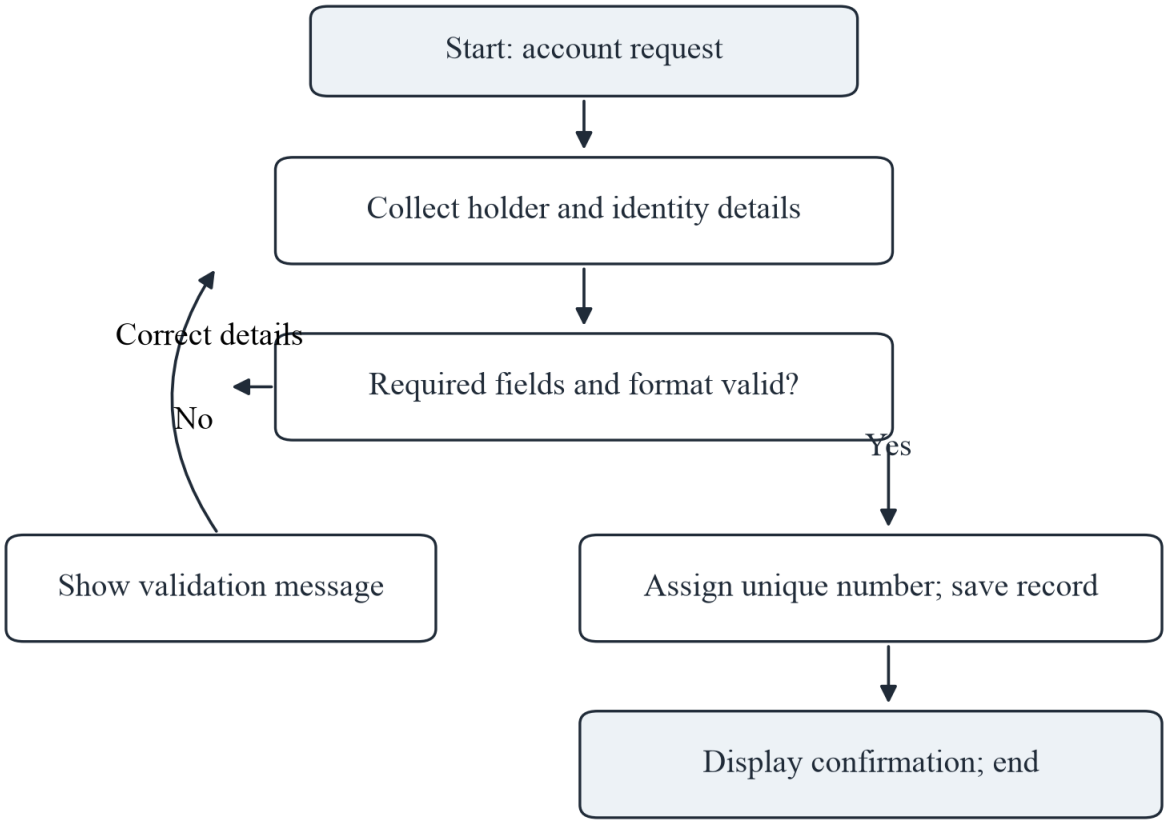


Figure 6.3: Conceptual account-opening workflow with a correction path for invalid details.

Required information includes the holder's name, an appropriate contact reference and the selected account type. The design should explain which fields are mandatory before confirmation. Empty or unclear information should return a correction message rather than creating an incomplete record.

The account number must be unique. In a future database, a key constraint could help enforce this rule. The opening balance should follow a stated policy and use an exact decimal representation. The new account should begin in a defined status.

The workflow separates checking from saving. This reduces the risk of creating a record that later fails basic validation. A confirmation should only be shown after the record is successfully established in a future implementation.

The diagram supports functional requirement FR-01 and the validation requirement FR-07. It does not claim that identity documents were collected or that any real account was opened for this report.

6.5 Deposit and Withdrawal Calculations

The financial rule in this case study is intentionally simple. A valid deposit adds the amount to the current balance. A valid withdrawal subtracts the amount after the account, status and available-balance checks succeed. No interest, fee or overdraft is included.

Table 6.2: Worked deposit and withdrawal calculations

Operation	Opening balance (INR)	Amount (INR)	Closing balance (INR)
Valid deposit	10,000.00	+ 2,500.00	12,500.00
Valid withdrawal	12,500.00	1,500.00	11,000.00
Second deposit	11,000.00	+ 3,000.00	14,000.00
Second withdrawal	14,000.00	800.00	13,200.00
Final deposit	13,200.00	+ 2,500.00	15,700.00
Invalid withdrawal	15,700.00	20,000.00 rejected	15,700.00 unchanged

The worked examples begin with INR 10,000. After deposits totaling INR 8,000 and permitted withdrawals totaling INR 2,300, the closing balance is INR 15,700. The calculation is 10,000 + 8,000 2,300 = 15,700.

The rejected withdrawal demonstrates an equally important rule. A request for INR 20,000 from a balance of INR 15,700 does not change the account. The correct closing value remains INR 15,700. A rejection should not create an ordinary debit entry.

A future implementation should use exact decimal arithmetic and should prevent the same request from being applied twice. The balance update and transaction record should succeed or fail together. These are design requirements, not experimentally verified properties here.

The examples support FR-03, FR-04 and FR-07. They also provide the values used in the later ledger and balance-trend chart. Keeping the same numbers across sections makes the report internally consistent and easy to check.

The amounts are fictional learning values. They are not evidence of customer activity, company revenue or system performance.

6.6 Withdrawal Decision Workflow

A withdrawal requires more care than a deposit because it reduces available funds. The conceptual workflow checks the account and amount before any balance change. The report assumes that withdrawals cannot create an overdraft.

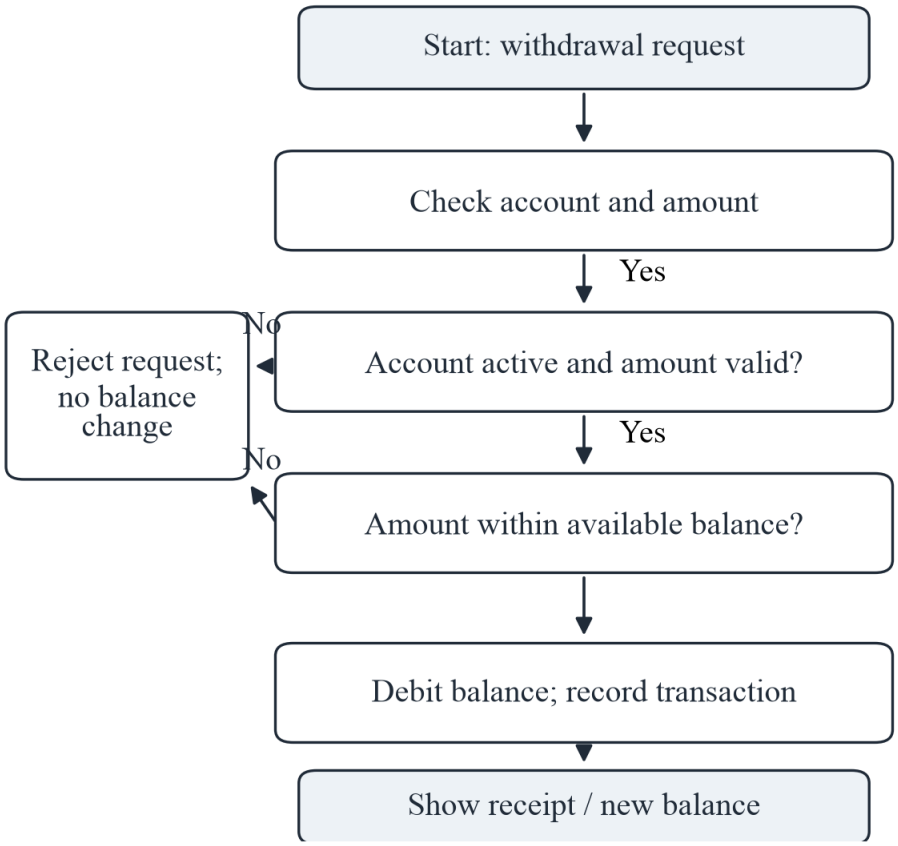


Figure 6.4: Withdrawal workflow with validation, available-balance checking and a no-change rejection path.

The first check identifies the account and confirms that the requested amount is a valid positive value. The next check considers whether the account is active and allowed to perform the operation. The available-balance check then compares the request with the permitted funds.

Only after these checks should the design reduce the balance and record the transaction. A receipt can show the accepted amount and resulting balance. If any check fails, the process returns a clear result without changing the financial state.

The order is important: checking after subtraction could create an invalid state. The diagram places decisions before the update.

This workflow supports FR-04 and FR-07. It also connects with Java control flow, validation and exception-aware design. A future implementation would need tests for unknown accounts, inactive accounts, invalid amounts, insufficient funds and interrupted saves.

The flowchart is a proposed process, not a trace from running software. It demonstrates the intended logic in a form that can be reviewed without code.

6.7 Balance Enquiry and Transaction History

Balance enquiry returns the current value for a selected account. Transaction history explains the entries that led to that value. Both are read-only operations and should not change the financial balance.

Table 6.3: Illustrative transaction ledger for sample account A101

Entry	Type	Amount (INR)	Balance after (INR)	Remark
T-000	Opening		10,000.00	Sample opening value
T-001	Deposit	2,500.00	12,500.00	Day 1
T-002	Withdrawal	1,500.00	11,000.00	Day 2
T-003	Deposit	3,000.00	14,000.00	Day 3
T-004	Withdrawal	800.00	13,200.00	Day 4
T-005	Deposit	2,500.00	15,700.00	Day 5

The ledger uses sample account A101 and an opening value of INR 10,000. Each successful entry shows the amount and balance after the operation. The closing value of INR 15,700 matches the worked calculations in Section 6.5.

Unique transaction identifiers help distinguish entries even when two amounts are equal. A future implementation should also store a timestamp and enough information to associate the entry with the correct account and authorised action.

History should be presented in a defined order, normally chronological. Filtering should prevent entries from another account appearing in the selected account's statement. A failed withdrawal should not appear as a successful debit; any operational error log would be separate from the financial ledger.

Balance enquiry is useful for a quick answer, while history supports explanation and checking. Together they address FR-05 and FR-06.

The table is manually prepared from the stated arithmetic. It is not exported from a database or banking application. Its purpose is to show how a clear record structure connects individual operations with the final account state.

6.8 Data Design and Entity Relationships

The conceptual data design contains three main entities: Customer, Account and Transaction. A customer can hold one or more accounts in the general model. Each account belongs to a customer. Each financial transaction belongs to one account.

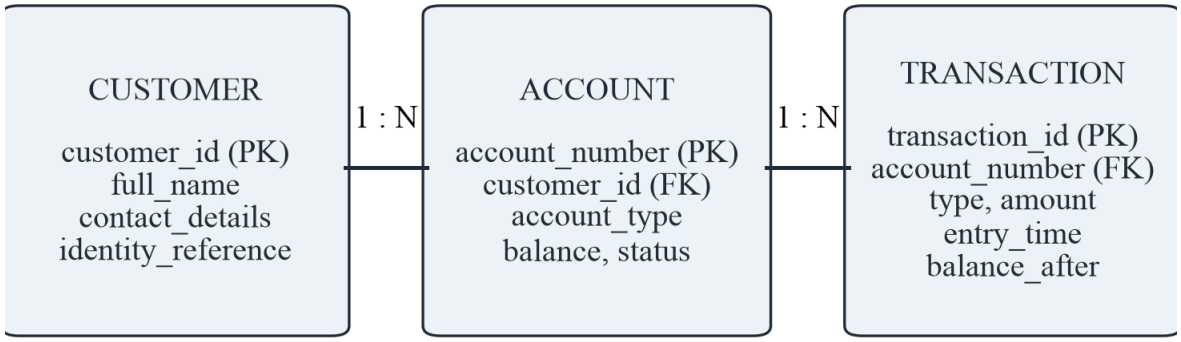


Figure 6.5: Conceptual entity relationship design with primary and foreign key relationships.

The customer entity stores identity-related information. The account entity stores the account number, customer reference, type, balance and status. The transaction entity stores a unique identifier, account reference, operation type, amount, time and resulting balance.

Primary keys distinguish records. Foreign keys connect related records. These relationships help prevent a transaction from referring to an undefined account. In a future database, constraints and transaction control would provide additional protection.

Sensitive identity information should be limited to what is necessary. The diagram uses an identity reference rather than storing full document details in the report. No real customer data are included.

The design is logical rather than tied to a particular database product. It could later be translated into tables using a relational database and accessed through JDBC. Exact column types, indexes and retention rules would require implementation decisions.

The model supports account lookup, history retrieval and traceable updates. It also separates the person who owns an account from the financial record and its individual events, reducing confusion in the later class design.

6.9 Class Design and Object Responsibilities

The class design translates the banking concepts into Java-oriented responsibilities. BankAccount represents account state and controlled operations. Transaction represents an individual financial event. AccountService coordinates use cases, and ValidationHelper centralises common checks.

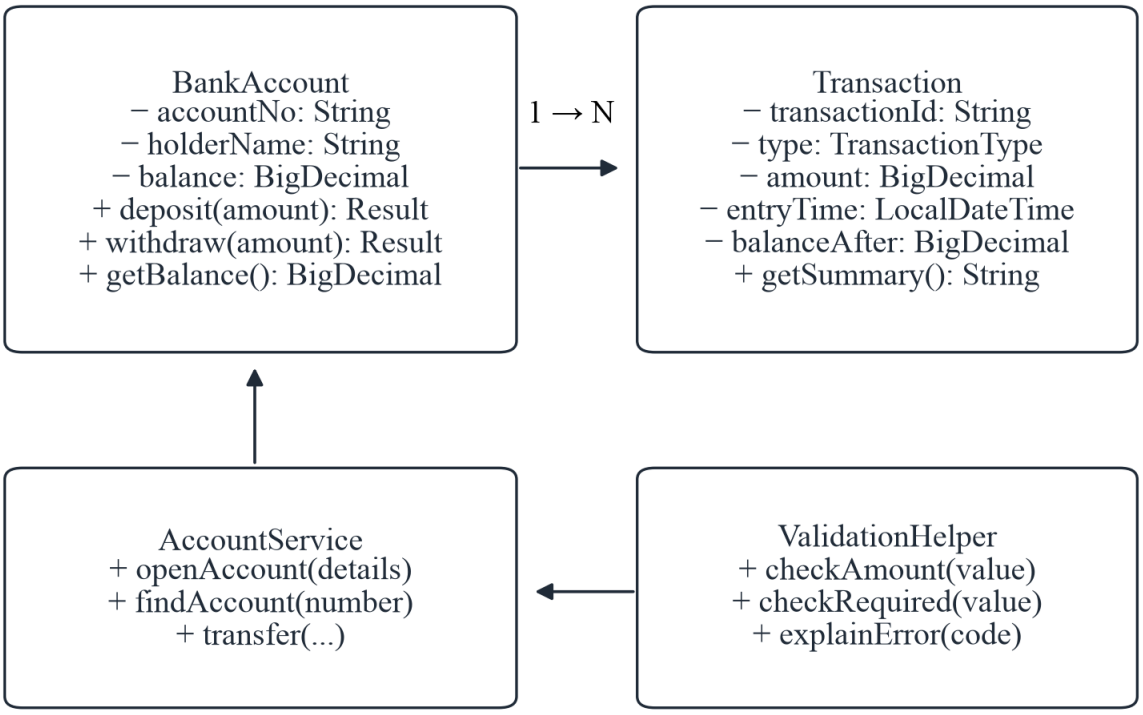


Figure 6.6: Conceptual class diagram showing proposed fields, methods and responsibilities.

BankAccount keeps its balance private in the proposed design. Deposit and withdrawal methods return a result rather than allowing unrestricted external assignment. The result can communicate success or a defined failure reason.

Transaction stores information about one event. It should not independently change an account balance. Association with an account explains where the event belongs.

AccountService coordinates lookup and use-case steps. ValidationHelper checks values and helps produce understandable messages. Presentation components would use these responsibilities rather than duplicating the rules.

BigDecimal is shown for money because exact decimal representation is important. LocalDateTime appears as a possible timestamp type; a real distributed system would need a clear time-zone and clock policy.

The diagram is a simplified academic model, not reverse-engineered source code. It omits constructors, persistence details and security components to keep attention on the main relationships. A future implementation would refine these details and test the behaviour.

6.10 Conceptual Dashboard Layout

The dashboard layout illustrates how a future interface could organise common account information. It is a static drawing prepared for the report, not a screenshot of a working application.

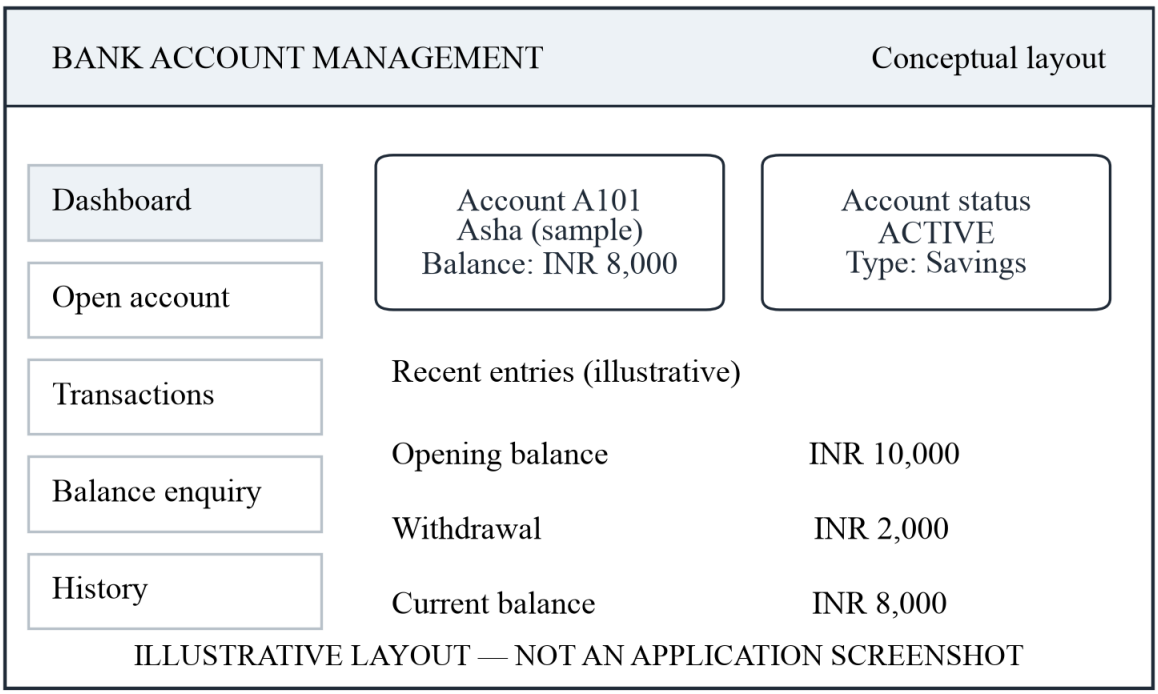


Figure 6.7: Illustrative dashboard layout; no running application or live account is represented.

The left area groups navigation choices. The main area shows the selected account, current balance and status. A recent-entry section provides a short view of activity. The layout separates identification, state and history so that the reader can find each type of information quickly.

Amounts should be labelled with a currency and formatted consistently. Status should use clear text rather than colour alone. Actions that change financial information should be visually distinct from read-only enquiries.

The sample account and balance are fictional. The layout does not demonstrate actual clicking, authentication, responsiveness or accessibility. Those qualities would require a prototype and user review.

This illustration addresses the notice's request for relevant visual project detail while respecting the chosen theory-only scope. It shows practical application of the design without falsely claiming implementation evidence.

6.11 Conceptual Transaction Form and Feedback

The transaction form illustrates the information needed for a deposit or withdrawal request. It includes an account identifier, operation type, amount and optional remark. The drawing is a conceptual layout and does not submit data.

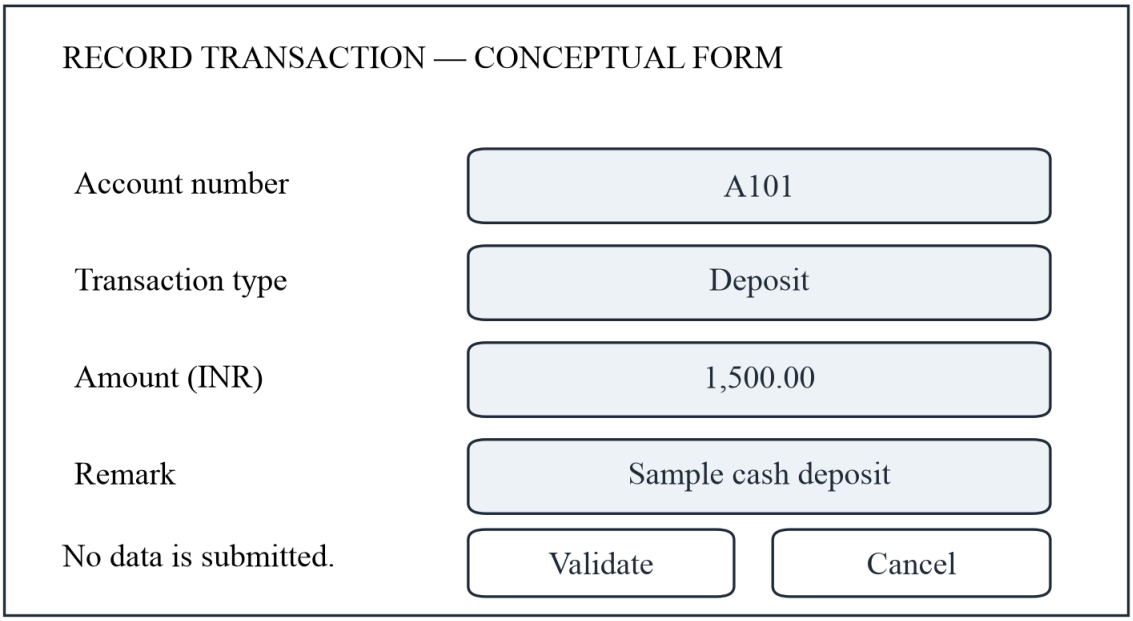


Figure 6.8: Illustrative transaction form with validation and cancellation choices.

The account identifier should be checked before the operation. The amount should be clearly labelled in INR and should accept the intended decimal format. The operation type should be unambiguous so that a deposit is not recorded as a withdrawal.

A validate or confirm action should lead to the business-rule checks described earlier. A cancel action should leave the account unchanged. After a successful future transaction, feedback should show the accepted operation and resulting balance. After rejection, feedback should explain the problem without exposing sensitive technical details.

The form should not encourage repeated submission of the same request. A future implementation would need duplicate protection and clear pending or completed states.

No usability experiment or interface testing was performed. The layout is included to make the proposed interaction understandable and to connect the requirements with a possible presentation.

6.12 Illustrative Dataset and Numerical Summary

The following dataset was created only for this report. It represents counts of three activity types over six illustrative months. It is not connected to a real bank, company, customer group or software log.

Table 6.4: Synthetic monthly activity dataset used in the charts

Month	Deposits	Withdrawals	Enquiries	Total
January	32	18	15	65
February	40	22	18	80
March	35	20	17	72
April	48	25	22	95
May	44	24	20	88
June	51	29	23	103
Total	250	138	115	503

The table contains 250 deposit entries, 138 withdrawal entries and 115 balance enquiries, giving 503 entries in total. The monthly totals are 65, 80, 72, 95, 88 and 103. Adding these values also gives 503, which provides a simple consistency check.

Deposits form the largest category. Their share is 250 divided by 503, or approximately 49.7 percent. Withdrawals form approximately 27.4 percent, and enquiries form approximately 22.9 percent. Percentages are rounded for display.

June has the highest total in this small dataset at 103 entries. March has 72 entries, fewer than February's 80, so the sequence is not a steady increase. This is a useful reminder that a chart should be read carefully rather than described as continuous growth.

The dataset supports practice with tables, addition, comparison and proportion. It does not support conclusions about customer behaviour, branch efficiency, fraud or financial performance.

Using a clearly labelled synthetic dataset allows the report to include graphs and charts without inventing experimental results. The next sections convert the same values into visual form and explain what can reasonably be observed.

6.13 Transaction-Volume Chart and Interpretation

The stacked bar chart represents the monthly counts from Table 6.4. Each bar's total is the sum of deposits, withdrawals and balance enquiries for that illustrative month.

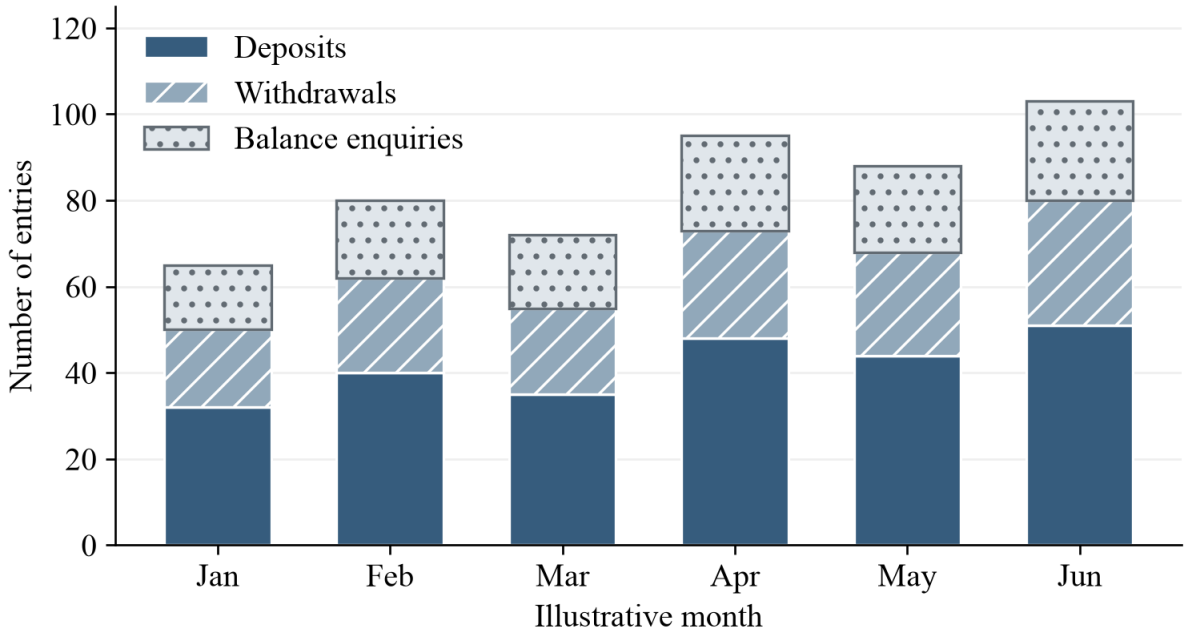


Figure 6.9: Monthly activity counts from the report's synthetic dataset; not measured system results.

The chart makes comparison easier. June has the tallest bar at 103 entries, while January has the shortest at 65. April and May are higher than the first three months, but March is lower than February. The visual pattern therefore includes variation rather than uninterrupted growth.

The segments show composition within each month. Deposits form the largest segment throughout the dataset. This agrees with the total of 250 deposits out of 503 entries.

A stacked chart is useful for comparing totals and broad composition, but exact segment comparison can be harder when the starting heights differ. The table remains important for checking precise values.

No response time, processing speed or productivity improvement is measured by this chart. The vertical axis counts fictional entries, not seconds, users or revenue.

The example demonstrates how a Java-oriented system could later present summary information, while the report itself uses manually prepared data and a static chart.

6.14 Transaction-Mix and Balance-Trend Charts

The pie chart shows the share of each activity type across the complete synthetic dataset. The line chart follows the separate A101 worked example from opening balance through five successful transactions.

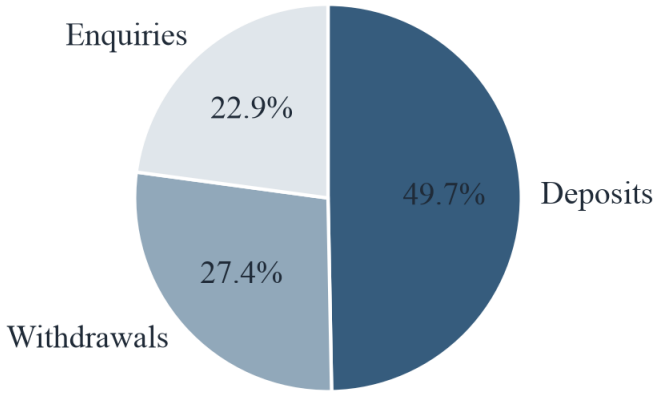


Figure 6.10: Proportions of deposits, withdrawals and enquiries in the synthetic activity dataset.

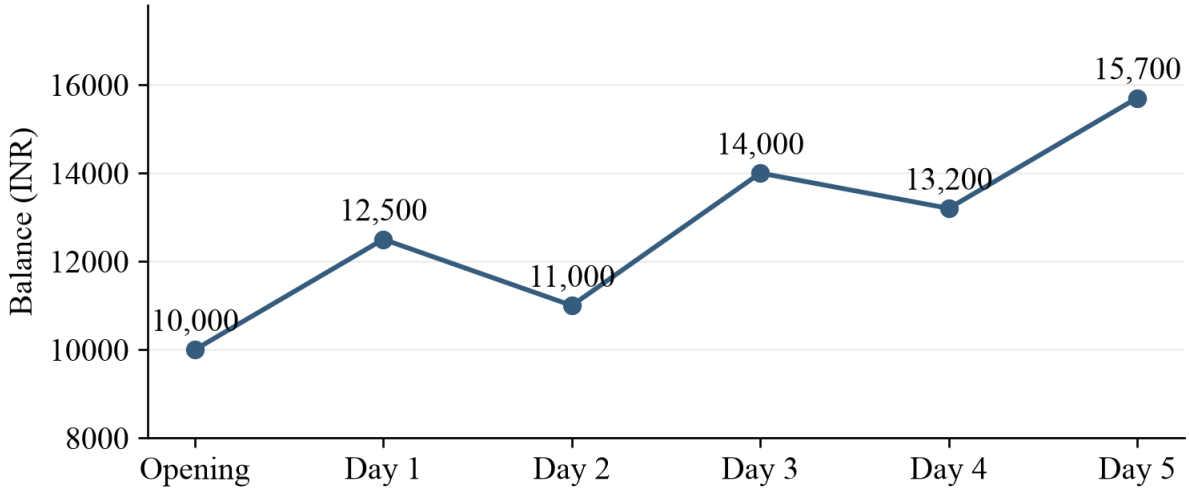


Figure 6.11: Balance trend calculated from the fictional A101 ledger in Table 6.3.

Deposits account for about 49.7 percent of activity, withdrawals about 27.4 percent and enquiries about 22.9 percent. The pie chart describes composition, not change over time.

The balance line begins at INR 10,000 and ends at INR 15,700. It falls after withdrawals and rises after deposits. The net increase is INR 5,700. This follows directly from the ledger and is not a forecast.

The two charts use different datasets and answer different questions. Keeping their captions and explanations separate prevents an activity-count summary from being mistaken for the balance history of one account.

6.15 Conceptual Testing, Security and Limitations

The review plan below describes checks that a future implementation should satisfy. It is not a record of executed tests. The present evidence consists of rules, arithmetic, diagrams and written reasoning.

Table 6.5: Conceptual validation and security review plan

ID	Check or test input	Expected behaviour	Evidence here
TC-01	Deposit INR 500 into A101.	Balance rises by exactly INR 500.	Arithmetic example.
TC-02	Deposit INR 100.	Reject; leave all records unchanged.	Rule specification.
TC-03	Withdraw more than available balance.	Reject with a clear message.	Withdrawal flowchart.
TC-04	Use unknown account A999.	Return account-not-found result.	Rule specification.
TC-05	Repeat the same transaction identifier.	Do not create a duplicate financial effect.	Design requirement.
TC-06	Attempt access without permission.	Deny access to account information.	Security discussion.
TC-07	Interrupt a future save operation.	Avoid a balance change without its transaction record.	Future test needed.

Security must be considered separately from normal validation. Authentication establishes who is using the system; authorisation decides what that user may do. Sensitive information should be protected in storage and transmission, and logs should avoid exposing unnecessary personal or financial details [10].

A private balance field supports encapsulation but does not provide these protections by itself. A real service would also need secure configuration, dependency maintenance, backup planning, duplicate-request control and recovery from interrupted updates.

The main limitation is the absence of a working model. No claim can be made about actual speed, concurrent users, database reliability, accessibility or resistance to attack. The interface drawings are not screenshots, the dataset is synthetic, and the stakeholder needs are reasoned assumptions rather than interview findings.

Within those limits, the report provides a traceable conceptual design. Each major operation has a rule, a workflow or a worked example, and the limitations identify what must be verified before any practical deployment.

CHAPTER 7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion and Summary of Learning

This report presented a simple conceptual study of a Bank Account Management System using Java. The project connected the student's one-month Java Programming training with a familiar information-processing problem. It explained the problem, defined objectives, described the training context, reviewed relevant technology and developed a bounded design.

The study showed how account information can be organised around customers, accounts and transactions. It explained why deposits and withdrawals change state while enquiries only read it. It also showed why validation should occur before a financial update and why a rejected request should leave the balance unchanged.

Java concepts were used where they supported the explanation. Classes and objects represented account structure and individual records. Encapsulation protected the balance behind controlled operations. Data-type discussion explained why exact decimal representation matters for money. Collections and JDBC were introduced as ways to organise and preserve related information.

The diagrams translated the written rules into visual form. The architecture separated responsibilities, the use-case view identified user goals, and the flowcharts described decision sequences. The data and class diagrams connected the domain with a possible Java implementation. The interface layouts illustrated presentation without pretending to be application screenshots.

The numerical examples developed basic analytical care. The A101 ledger produced a closing balance of INR 15,700 from the stated operations. The separate activity dataset supported totals, percentages and charts. All values were clearly identified as fictional learning data.

The most important limitation is also clear: no working banking model was developed, connected or tested for this report. The work demonstrates design understanding, not operational reliability or measured improvement. Security and persistence were discussed as requirements for future work rather than completed features.

The project therefore meets its academic purpose within the selected scope. It provides a readable, consistent and honest application of Java theory to a relevant case study, while leaving implementation, verification and real-world integration as separate future tasks.

7.2 Limitations and Future Scope

The present report is limited to theory, conceptual design and illustrative analysis. Its next stage would be a small supervised prototype using fictional data. That prototype should be developed only after the rules and requirements have been reviewed.

Table 7.1: Possible future extensions beyond the present theory report

Extension	What it would add	What must be checked first
Working Java prototype	Actual screens and stored records under supervision.	Correctness, safe test data and faculty approval.
Database integration	Persistent accounts and transaction history.	Keys, constraints and backup behaviour.
Authentication and roles	Different access for customers and staff.	Identity checks and permission boundaries.
Transfer between accounts	A coordinated debit and credit.	Atomic update and rollback behaviour.
Statement export	A printable account statement.	Date filtering and privacy controls.
Accessibility review	Improved keyboard use and readable messages.	Testing with representative users.

A first implementation could focus on one account type and the basic operations already described. It should use exact decimal arithmetic, clear validation messages and a durable record of successful transactions. Tests should include invalid inputs, insufficient balance and duplicate requests.

Database integration would then allow account and transaction information to survive restarts. Keys, constraints and transaction control would need particular attention. A balance update and its matching entry should not be saved independently.

Authentication and role-based access would become necessary before more than one user role is represented. A customer should not gain clerk permissions, and sensitive information should not appear in unrestricted logs or exports.

Later extensions could include transfers, statement export and accessibility review. Each extension adds rules and should be tested separately. A transfer, for example, requires a coordinated debit and credit rather than two unrelated updates.

The report's other limitation is evidence. No user study, performance benchmark or company dataset was used. Future claims about usability, speed or reliability would require appropriate measurement and transparent reporting.

This staged scope keeps the project simple today while providing a sensible path for further learning. The current submission remains a theory-based report, and the table describes possibilities rather than completed work.

CHAPTER 8 REFERENCES

8.1 References and Source Notes

[1] Oracle, The Java Tutorials: About the Java Technology. https://docs.oracle.com/javase/tutorial/getStarted/intro/definition.html

[2] Oracle, The Java Tutorials: Object-Oriented Programming Concepts. https://docs.oracle.com/javase/tutorial/java/concepts/

[3] Oracle, Java SE 21 API: BigDecimal. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/math/BigDecimal.html

[4] Oracle, The Java Tutorials: Exceptions. https://docs.oracle.com/javase/tutorial/essential/exceptions/

[5] Oracle, The Java Tutorials: Collections Framework Overview. https://docs.oracle.com/javase/tutorial/collections/intro/

[6] Oracle, The Java Tutorials: JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/basics/

[7] Oracle, The Java Tutorials: Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/

[8] J. Bloch, Effective Java, 3rd edition. Addison-Wesley Professional, 2018.

[9] K. Sierra, B. Bates and T. Gee, Head First Java, 3rd edition. O Reilly Media, 2022.

[10] OWASP Foundation, Input Validation Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

[11] EWB EDUTECH PRIVATE LIMITED, Internship Offer Letter, reference EWB-Java-Programming-2026-9111, dated 15 June 2026. Student-supplied document.

[12] EWB Courses, Internship Completion Certificate, certificate EWB/INT/2026/9765, issued 16 July 2026. Student-supplied document.

[13] Department of Computer Science and Engineering, University of Lucknow, Submission of B.Tech CSE Summer Internship Report, notice supplied by the student, including formatting, technology-based training and submission instructions.

[14] University of Lucknow, B.Tech CSE Summer Training Cover Page Format. https://udrc.lkouniv.ac.in//Content/DepartmentContent/SY_ebdfbfee-68a3-4a45-b40f-753d7d7b0f6b_58.pdf

Source note: official documentation supports the technology explanations; the supplied offer, certificate, notice and cover format support the training and submission context. The banking problem, names, account numbers, interface drawings and numerical datasets are original academic illustrations prepared for this report. They are not company records or results from a working application.